



INSTITUT
POLYTECHNIQUE
DE PARIS

Rapport SIM202

Simulation gravitationnelle

Jude Nemer KHAZZAKA

Ayoub ED-DIB

Léto VAN RUYMBEKE

Mars 2026

Table des matières

	1
1 Introduction générale	2
2 Objectifs du projet	2
3 Rappel du modèle théorique	3
4 Architecture générale du projet	3
5 Gestion des particules	3
5.1 Fonctions principales de <code>Particule.cpp</code>	3
5.2 Rôle de <code>creerSystemePlummer</code>	4
6 Structure hiérarchique des boîtes	4
6.1 Fonctions principales de <code>Boite.cpp</code>	5
7 Calcul des forces : approche de type Barnes–Hut	5
8 Évolution dynamique et schéma saute-mouton	5
9 Mise à jour dynamique de l’arbre	6
10 Visualisation 2D : rôle et intérêt	6
10.1 Fonctions utiles dans <code>test2D.cpp</code>	6
11 Initialisation des galaxies selon le modèle de Plummer	8
12 Collision de deux galaxies en 3D	9
12.1 Fonctions utiles dans <code>test3D.cpp</code>	9
13 Méthodologie suivie	12
14 Validation du projet	12
15 Analyse critique du projet	13
16 Perspectives d’amélioration	13
17 Conclusion	13
18 Annexe : fichiers principaux du projet	14

1 Introduction générale

La simulation gravitationnelle constitue un problème classique de physique numérique et de calcul scientifique. Lorsqu'un grand nombre de particules interagissent entre elles sous l'effet de la gravitation, le calcul exact de toutes les interactions devient rapidement trop coûteux. En effet, chaque particule subit l'influence de toutes les autres, ce qui entraîne un nombre de calculs qui croît très vite avec la taille du système.

Le sujet proposé consiste à simuler un ensemble de particules ponctuelles de masse donnée en interaction gravitationnelle classique. Une application naturelle de ce type de modèle est l'étude de systèmes astrophysiques, en particulier la collision de deux galaxies. L'objectif du projet n'est donc pas uniquement de produire une animation graphique, mais de construire une architecture logicielle capable de représenter, d'organiser et de faire évoluer un système N-corps de manière efficace.

Afin de réduire le coût des calculs, le sujet repose sur un modèle à deux échelles : les particules proches sont traitées de manière plus précise, tandis que les particules lointaines sont regroupées au sein de boîtes hiérarchiques et remplacées par une masse équivalente située en leur barycentre. Cette idée mène naturellement à une structure arborescente de l'espace, de type quadtree en deux dimensions et octree en trois dimensions.

Dans notre projet, nous avons mis en place :

- une classe `Particule` représentant les corps du système ;
- une classe `Boîte` gérant la structure hiérarchique de l'espace ;
- un calcul approché des forces inspiré de l'algorithme de Barnes–Hut ;
- une évolution temporelle par schéma saute-mouton ;
- une mise à jour dynamique de l'arbre au cours du temps ;
- une visualisation 2D pour la validation des mécanismes fondamentaux ;
- une visualisation 3D étendue à la collision de deux galaxies initialisées selon le modèle de Plummer.

2 Objectifs du projet

Le projet poursuit plusieurs objectifs complémentaires.

Le premier est de représenter correctement les particules du système, avec leur position, leur vitesse, leur masse et la force qui s'exerce sur elles. Chaque particule doit également pouvoir être reliée aux autres dans une structure de parcours simple, afin de manipuler efficacement l'ensemble du système.

Le deuxième objectif est de construire une structure hiérarchique de boîtes permettant de répartir les particules dans l'espace. Cette structure doit rendre possible un calcul plus rapide des forces gravitationnelles, tout en conservant une bonne cohérence physique.

Le troisième objectif est de mettre à jour les positions et les vitesses dans le temps à l'aide d'un schéma numérique stable, ici le schéma saute-mouton demandé dans l'énoncé.

Enfin, un dernier objectif est de rendre le comportement du système observable grâce à une visualisation graphique. La version 2D sert principalement à comprendre la structure hiérarchique et à valider les mécanismes élémentaires, tandis que la version 3D permet de représenter un système plus proche d'un cadre astrophysique et d'y implémenter une collision entre deux galaxies.

3 Rappel du modèle théorique

Dans un système gravitationnel classique, la force exercée sur une particule i est la somme des forces dues à toutes les autres particules j . Une approche exacte impose donc de considérer explicitement toutes les paires de particules. Si le nombre de corps devient important, cette stratégie devient rapidement trop coûteuse.

Le sujet propose alors une idée simple mais très efficace : utiliser la loi exacte pour les particules proches et une loi approchée pour les particules lointaines. Lorsqu'un groupe de particules est suffisamment éloigné, on peut remplacer ce groupe par une masse unique placée en son centre de masse. Cette approximation permet de réduire considérablement le nombre d'interactions calculées.

Cette logique s'accompagne d'une structuration hiérarchique de l'espace en boîtes. Chaque boîte peut contenir plusieurs sous-boîtes si plusieurs particules y sont présentes, ou bien contenir directement une particule si elle est terminale. L'espace est donc organisé de manière récursive, ce qui fournit une base naturelle pour le calcul approché des forces.

4 Architecture générale du projet

Le projet repose sur trois blocs principaux :

- la classe `Particule` ;
- la classe `Boite` ;
- les programmes de test `test2D.cpp` et `test3D.cpp`.

La classe `Particule` gère les corps du système : position, vitesse, force, masse et historique des positions. La classe `Boite` gère la structure hiérarchique de l'espace, la subdivision, le centre de masse et le calcul des forces. Les deux fichiers de test servent à faire tourner la simulation et à l'afficher :

- en 2D pour valider la structure ;
- en 3D pour visualiser et simuler la collision de deux galaxies.

Cette séparation rend le projet plus propre, plus lisible et plus facile à faire évoluer.

5 Gestion des particules

La classe `Particule` représente les entités physiques de base de la simulation. Chaque particule possède :

- une position ;
- une vitesse ;
- une force ;
- une masse ;
- un pointeur vers la particule suivante ;
- une liste des positions successives.

Les particules sont stockées dans une liste chaînée circulaire, ce qui permet de parcourir facilement l'ensemble du système.

5.1 Fonctions principales de `Particule.cpp`

Voici le rôle des fonctions principales liées aux particules.

- `Particule(...)` : crée une particule et initialise toutes ses grandeurs physiques.

- `getMasse()` : retourne la masse de la particule.
- `getPosition()` : retourne la position actuelle.
- `getVitesse()` : retourne la vitesse actuelle.
- `getForce()` : retourne la force actuelle.
- `getSuivante()` : retourne la particule suivante dans la liste.
- `deplacer(...)` : change la position de la particule.
- `ajouter_position(...)` : ajoute une position dans l'historique.
- `saute_mouton(...)` : met à jour la vitesse et la position selon le schéma saute-mouton.
- `print()` : affiche les informations d'une particule.
- `print_liste()` : affiche toute la liste des particules.
- `forceEntreParticules(...)` : calcule la force gravitationnelle entre deux particules.
- `forceTotaleSurParticule(...)` : additionne les forces exercées sur une particule par toutes les autres.
- `creerSysteme(...)` : crée un système simple avec des positions aléatoires.
- `detruireSysteme(...)` : libère la mémoire du système.
- `creerSystemePlummer(...)` : crée un système initialisé selon le modèle de Plummer.

5.2 Rôle de `creerSystemePlummer`

Cette fonction est importante car elle permet de construire une galaxie autogravitante plus réaliste qu'un simple nuage aléatoire. Elle effectue les étapes suivantes :

1. tirage du rayon de chaque particule selon la loi de Plummer ;
2. choix d'une direction aléatoire pour la position ;
3. calcul d'une vitesse compatible avec le potentiel de Plummer ;
4. choix d'une direction aléatoire pour la vitesse ;
5. création de la particule avec une masse uniforme.

6 Structure hiérarchique des boîtes

La structure hiérarchique de boîtes constitue le cœur algorithmique du projet. Elle permet de répartir les particules dans l'espace et de construire une organisation récursive cohérente avec le modèle à deux échelles.

L'ajout d'une particule dans l'arbre suit une logique récursive :

1. si la boîte est terminale et vide, la particule y est placée directement ;
2. si la boîte est terminale mais déjà occupée, elle est subdivisée en sous-boîtes et les particules sont redistribuées ;
3. si la boîte possède déjà des sous-boîtes, la particule est dirigée vers la sous-boîte qui contient sa position.

À chaque insertion, la masse totale et le centre de masse des boîtes concernées sont mis à jour. Cette mise à jour est fondamentale, car ces grandeurs sont ensuite utilisées dans l'approximation des interactions gravitationnelles.

6.1 Fonctions principales de Boite.cpp

Voici le rôle des fonctions importantes de la classe `Boite`.

- `Boite(...)` : construit une boîte avec son niveau, son centre et sa taille.
- `~Boite()` : détruit récursivement les boîtes filles et sœurs.
- `estTerminale()` : dit si la boîte n'a pas de filles.
- `estVide()` : dit si la boîte ne contient ni particule ni sous-boîte.
- `ajouterParticule(...)` : insère une particule dans l'arbre.
- `supprimerParticule(...)` : retire une particule de la structure.
- `diviserBoite()` : subdivise une boîte en sous-boîtes.
- `getIndiceSousBoite(...)` : détermine dans quelle sous-boîte doit aller une particule.
- `mettreAJourCentreMasse()` : recalcule la masse totale et le centre de masse.
- `calculerForces(...)` : calcule la contribution gravitationnelle d'une boîte sur une particule.
- `nettoyerBoitesVides()` : supprime les sous-boîtes inutiles.
- `contient(...)` : vérifie si une position appartient à la boîte.
- `trouverBoiteTerminale(...)` : retrouve la boîte terminale qui contient une position.
- `recupererParticulesDeplacees(...)` : repère les particules sorties de leur boîte.
- `mettreAJourArbre()` : remet à jour toute la structure après déplacement.

7 Calcul des forces : approche de type Barnes–Hut

Le calcul des forces gravitationnelles est réalisé dans la méthode `calculerForces` de la classe `Boite`. Cette méthode applique la logique suivante.

Si la boîte est vide, elle n'apporte aucune contribution. Si elle est terminale et contient une particule distincte de la particule cible, on calcule alors une interaction directe particule à particule.

Si la boîte n'est pas terminale, on évalue un critère fondé sur la taille de la boîte et la distance entre la particule cible et le centre de masse de la boîte. Lorsque ce critère est suffisamment petit, la boîte est considérée comme suffisamment lointaine pour être remplacée par une masse ponctuelle située en son centre de masse. Dans le cas contraire, la boîte est ouverte récursivement et ses filles sont examinées.

Cette logique correspond à une approche de type Barnes–Hut. Elle permet d'accélérer la simulation tout en conservant une bonne approximation des interactions gravitationnelles globales.

8 Évolution dynamique et schéma saute-mouton

Pour faire évoluer le système dans le temps, nous utilisons le schéma saute-mouton indiqué dans le sujet. Ce schéma consiste à mettre à jour les vitesses sur un demi-pas de temps, puis les positions sur un pas complet.

Concrètement, pour chaque particule :

- la vitesse est d'abord modifiée en fonction de la force divisée par la masse ;
- la position est ensuite mise à jour ;
- la vitesse est enfin à nouveau ajustée sur un demi-pas.

Ce schéma présente l'avantage d'être plus stable qu'une méthode d'Euler explicite simple, notamment pour les systèmes dynamiques gravitationnels. Le pas de temps reste constant dans l'ensemble du projet.

9 Mise à jour dynamique de l'arbre

Après la mise à jour des positions, certaines particules peuvent sortir de leur boîte actuelle. Il est donc nécessaire de remettre à jour la structure hiérarchique.

La méthode `mettreAJourArbre` procède en plusieurs étapes :

1. elle identifie les particules qui ne sont plus contenues dans leur boîte terminale ;
2. elle les retire de leur ancienne position dans l'arbre ;
3. elle les réinsère dans les boîtes appropriées ;
4. elle recalcule ensuite les centres de masse.

Ce mécanisme garantit que la structure des boîtes reste cohérente avec la répartition spatiale réelle des particules tout au long de la simulation.

10 Visualisation 2D : rôle et intérêt

La version 2D du projet a principalement été utilisée comme outil de validation et de compréhension. Elle permet d'observer :

- la subdivision hiérarchique de l'espace sous forme de rectangles verts ;
- les particules sous forme de cercles rouges ;
- le comportement général de la dynamique au fil des itérations.

Le programme 2D calcule les forces via l'algorithme de Barnes–Hut, met à jour les positions avec le schéma saute-mouton, applique une correction simple pour éviter certains chevauchements visuels, puis met à jour l'arbre avant l'affichage.

10.1 Fonctions utiles dans `test2D.cpp`

- `dessinerGrille(...)` : dessine les boîtes de la structure hiérarchique.
- `dessinerParticules(...)` : dessine les particules rouges dans la fenêtre.
- la boucle principale : calcule les forces, met à jour les particules, corrige les chevauchements visuels, remet à jour l'arbre puis affiche le tout.

Cette version a donc joué un rôle essentiel dans la validation des mécanismes fondamentaux : construction de l'arbre, calcul des forces, évolution dynamique et mise à jour spatiale.

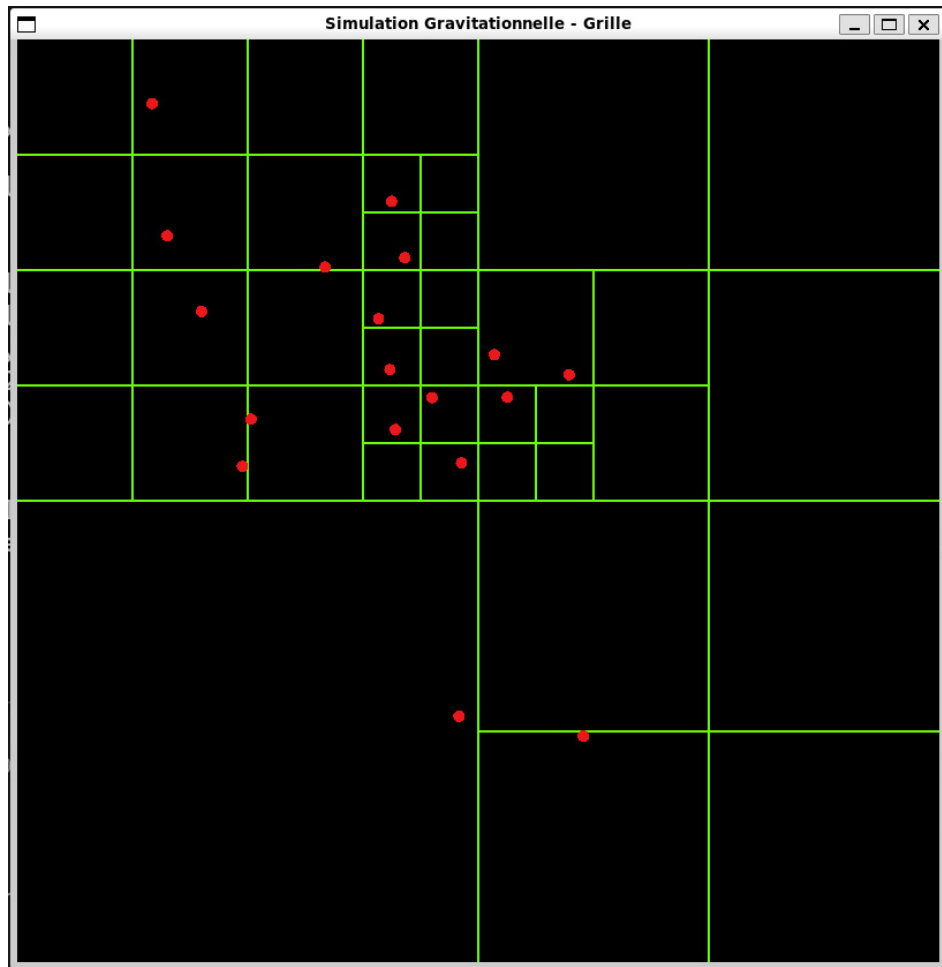


FIGURE 1 – **Vue générale de la simulation 2D.** Les particules sont représentées en rouge tandis que la structure hiérarchique des boîtes apparaît en vert.

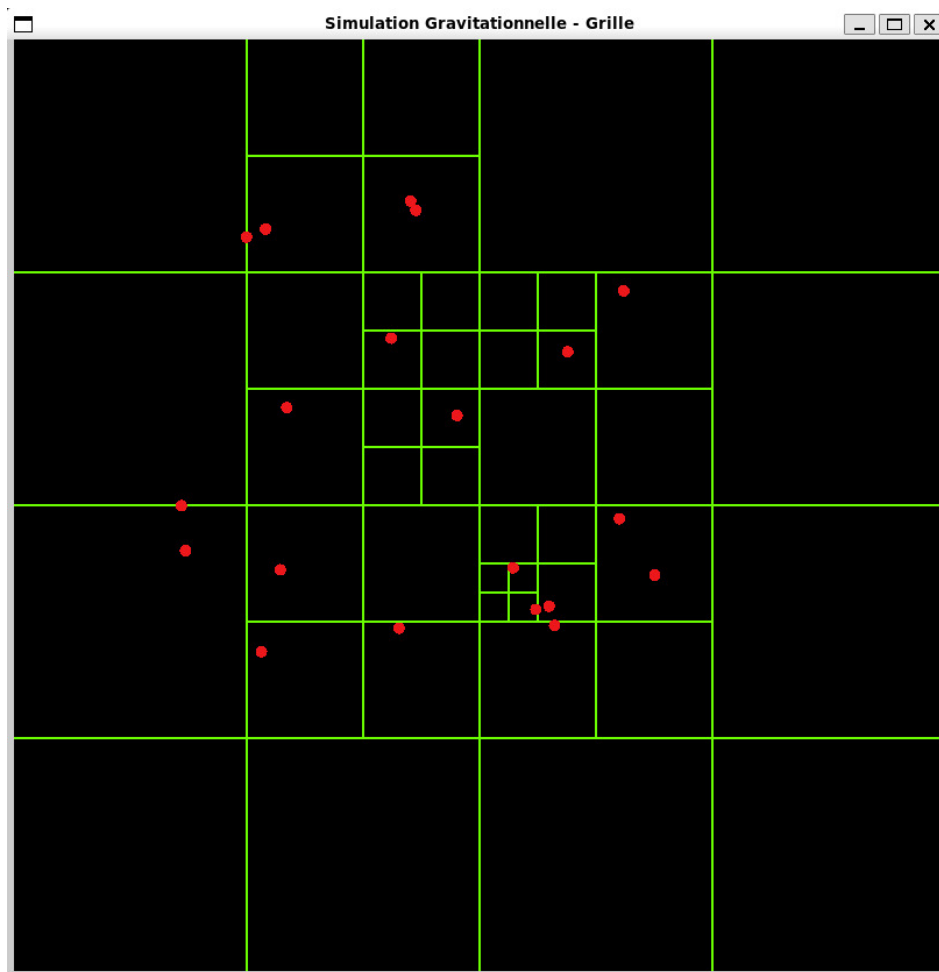


FIGURE 2 – **Zoom sur une zone subdivisée.** Cette figure met en évidence le raffinement local de la structure spatiale dans les régions les plus denses.

11 Initialisation des galaxies selon le modèle de Plummer

Afin de construire un système autogravitant plus réaliste, nous avons implémenté une génération de particules selon le modèle de Plummer, comme proposé dans l'annexe du sujet.

Le principe est le suivant :

- un rayon est tiré aléatoirement selon la loi de Plummer ;
- une direction aléatoire est ensuite choisie pour obtenir la position dans l'espace ;
- la norme de la vitesse est calculée à partir de la vitesse d'échappement et d'un tirage par rejet ;
- enfin, une direction aléatoire est choisie pour orienter le vecteur vitesse.

Chaque particule est créée avec une masse uniforme égale à $1/N$, conformément à la simplification proposée dans le sujet.

Cette initialisation permet de produire une galaxie sphérique autogravitante plus cohérente qu'un simple nuage uniforme de particules.

12 Collision de deux galaxies en 3D

L'application la plus avancée du projet consiste à simuler la collision de deux galaxies.

Pour cela, deux systèmes distincts de Plummer sont générés. Chacun est ensuite translaté dans l'espace afin de placer les deux galaxies à une certaine distance initiale. Une vitesse d'ensemble opposée est enfin ajoutée à chacune des galaxies afin de provoquer leur rapprochement et leur collision.

Les deux listes circulaires de particules sont ensuite fusionnées en un seul système global. La simulation se poursuit alors exactement selon le même schéma que pour un système simple :

1. calcul des forces ;
2. intégration temporelle ;
3. mise à jour de l'arbre ;
4. affichage.

12.1 Fonctions utiles dans `test3D.cpp`

Voici le rôle des fonctions les plus importantes du fichier `test3D.cpp`.

- `toRaylibVector(...)` : convertit les coordonnées du projet en coordonnées affichables dans Raylib.
- `dessinerOctree(...)` : dessine récursivement l'octree en 3D.
- `dessinerParticules3D(...)` : dessine toutes les particules sous forme de sphères rouges.
- `calculerToutesLesForces(...)` : remet les forces à zéro puis calcule les forces sur toutes les particules.
- `integrerSysteme(...)` : fait évoluer toutes les particules dans le temps.
- `insérerSystemeDansArbre(...)` : insère toutes les particules dans la boîte racine.
- `preparerGalaxie(...)` : translate une galaxie, lui ajoute une vitesse d'ensemble et ajuste la masse de ses particules.
- `fusionnerGalaxies(...)` : relie deux listes circulaires pour former un seul système global.

Cette approche répond à l'application proposée par le sujet, tout en exploitant naturellement la structure 3D du modèle de Plummer.

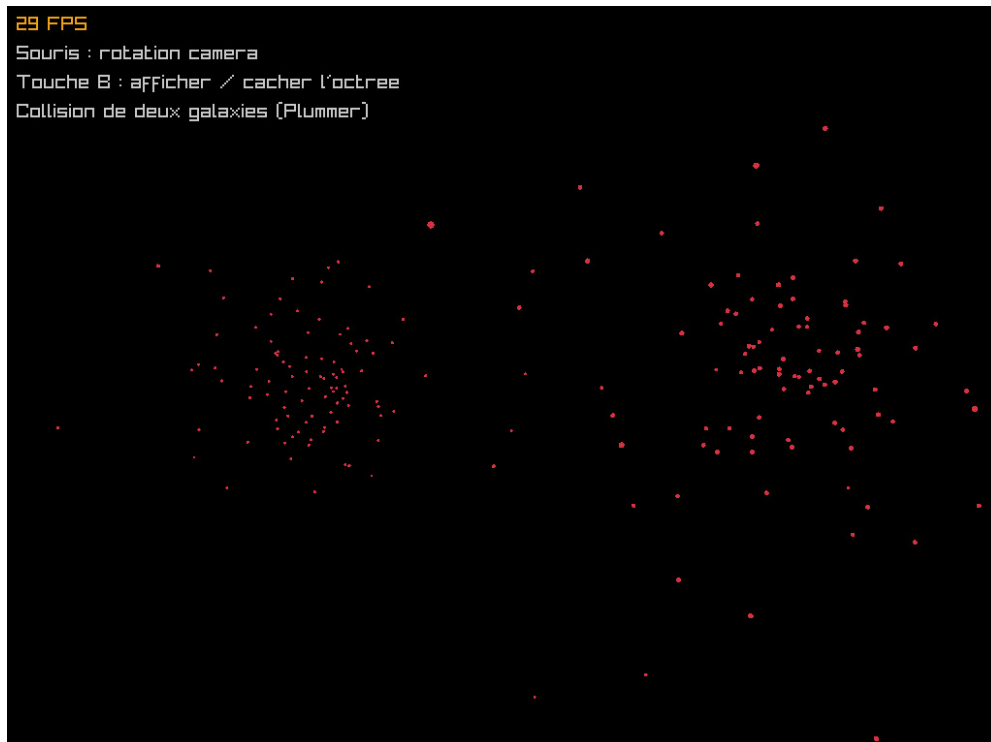


FIGURE 3 – **Début de la simulation 3D** : Les deux galaxies sont initialement séparées dans l'espace avant leur rapprochement.

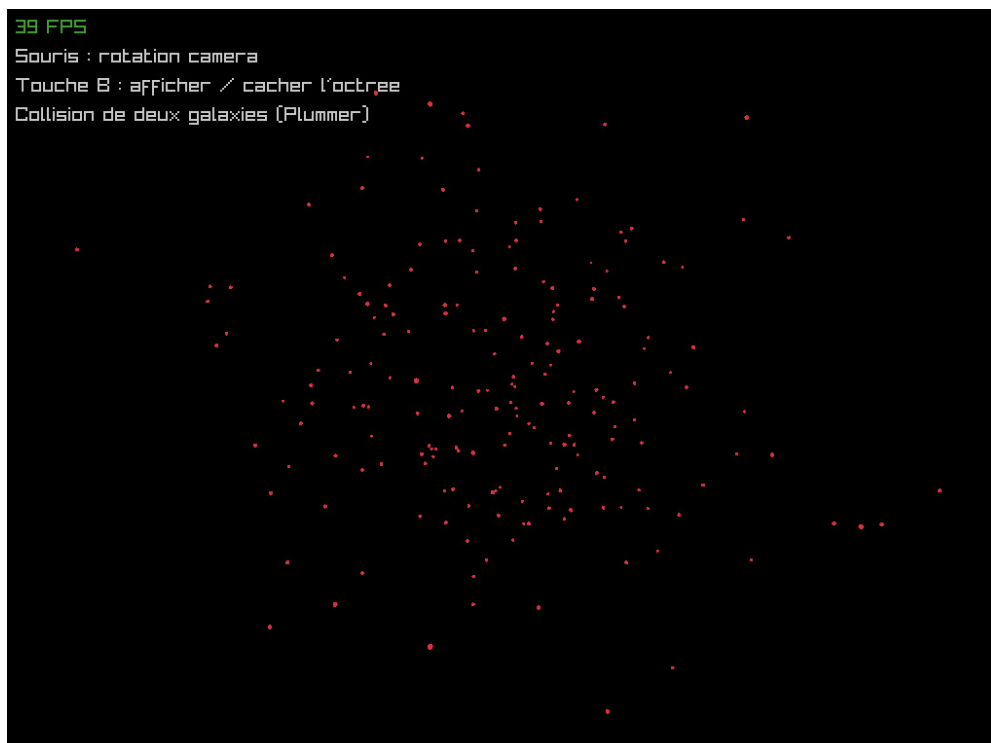


FIGURE 4 – **Phase de collision** : Les deux systèmes entrent en collision et leur structure se déforme progressivement.

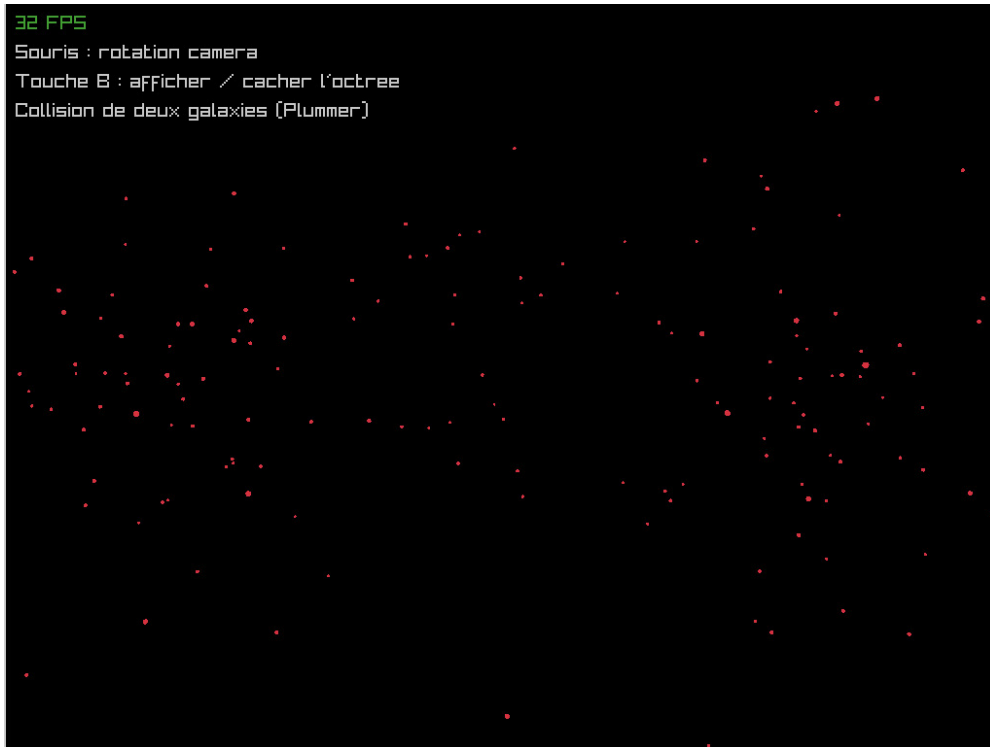


FIGURE 5 – **Phase post-collision** : Les deux galaxies se séparent progressivement et leurs structures restent perturbées par l'interaction gravitationnelle.

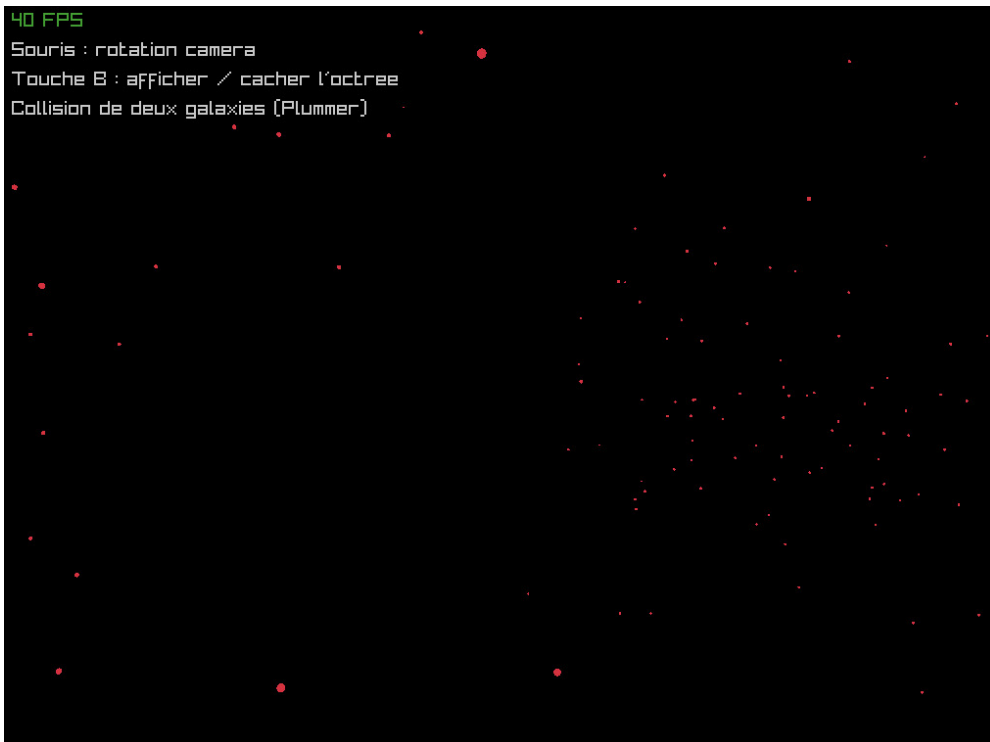


FIGURE 6 – **Phase finale** : Les deux galaxies sont complètement séparées et retrouvent progressivement un état d'équilibre décrit par le modèle de Plummer.

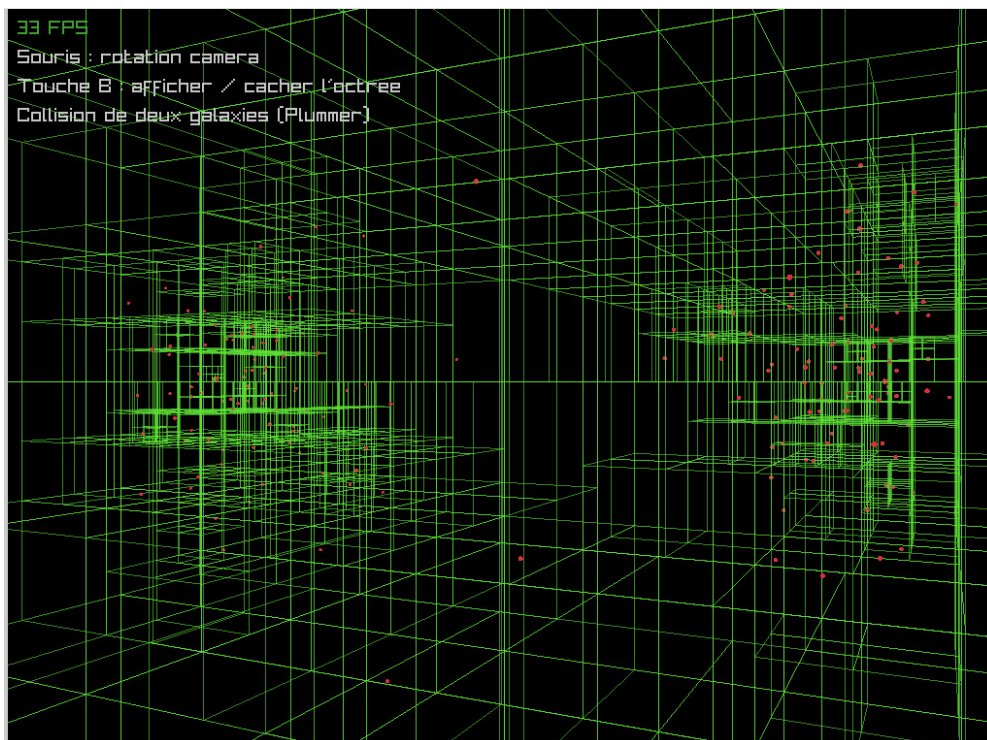


FIGURE 7 – **Affichage de l’octree en 3D.** Les cubes verts représentent la structure hiérarchique utilisée pour organiser les particules dans l’espace et accélérer le calcul des interactions.

13 Méthodologie suivie

Le travail a été conduit de manière progressive.

Dans un premier temps, nous avons construit les classes de base représentant les particules et les boîtes. L’objectif était de disposer d’une architecture correcte avant de chercher à simuler des cas complexes.

Dans un second temps, nous avons implémenté les principales fonctionnalités : insertion de particules, subdivision des boîtes, calcul des forces, mise à jour des positions et remise à jour de l’arbre.

La visualisation 2D a ensuite servi d’outil de validation intermédiaire. Elle a permis de vérifier que la structure hiérarchique fonctionnait correctement et que l’évolution du système restait cohérente.

Enfin, nous avons étendu le projet à la 3D et à la collision de deux galaxies initialisées selon Plummer, ce qui constitue l’aboutissement logique du travail demandé.

14 Validation du projet

La validation du projet a été réalisée principalement de manière visuelle et structurelle.

D’une part, la version 2D permet de vérifier que la subdivision de l’espace est cohérente, que les particules sont bien localisées dans leurs boîtes respectives et que l’arbre se met correctement à jour au cours du temps.

D’autre part, la version 3D permet de vérifier que deux galaxies peuvent être générées, placées dans l’espace, puis mises en mouvement l’une vers l’autre dans un scénario de

collision.

Cette validation est cohérente avec la méthodologie indiquée dans le sujet, qui insiste sur la nécessité de valider les différentes fonctionnalités sur des cas simples avant de passer à une application plus avancée.

15 Analyse critique du projet

Le projet présente plusieurs points forts.

Le premier est la cohérence de son architecture. La séparation entre la classe `Particule`, la classe `Boite` et les programmes de test rend le code plus lisible et plus facile à faire évoluer.

Le deuxième est la mise en œuvre réelle d'un calcul approché de type Barnes–Hut. Le projet ne repose donc pas sur une simple simulation naïve, mais sur une stratégie d'optimisation adaptée au problème N-corps.

Le troisième est l'extension vers une collision de galaxies en 3D, qui donne au projet une application plus concrète et plus proche du cadre astrophysique présenté dans le sujet.

Certaines limites peuvent néanmoins être relevées :

- le choix du paramètre ε d'adoucissement pourrait être approfondi ;
- l'influence de θ sur le compromis précision / performance n'a pas été étudiée en détail ;
- une comparaison quantitative entre calcul exact et calcul approché pourrait enrichir le projet.

16 Perspectives d'amélioration

Plusieurs pistes peuvent être envisagées pour prolonger le travail :

- étudier plus rigoureusement l'effet du paramètre θ ;
- comparer les temps de calcul avec et sans approximation ;
- enrichir la visualisation par des trajectoires complètes ;
- rendre les conditions initiales plus facilement configurables ;
- explorer des collisions de galaxies plus complexes ou asymétriques.

17 Conclusion

En conclusion, ce projet répond de manière solide aux objectifs essentiels du sujet. Il combine une représentation physique des particules, une structure hiérarchique de l'espace, un calcul approché des forces, un schéma numérique d'évolution temporelle et une visualisation graphique en deux et en trois dimensions.

La version 2D a joué un rôle important dans la validation des mécanismes fondamentaux du projet. La version 3D a permis d'aller plus loin, en implémentant une collision entre deux galaxies initialisées selon le modèle de Plummer.

Le projet constitue ainsi une base claire, cohérente et extensible pour l'étude de systèmes N-corps gravitationnels.

18 Annexe : fichiers principaux du projet

Les fichiers principaux du projet sont les suivants :

- `Particule.hpp` et `Particule.cpp` : définition des particules, intégration temporelle et génération de systèmes ;
- `Boite.hpp` et `Boite.cpp` : structure hiérarchique, calcul des forces et mise à jour de l'arbre ;
- `test2D.cpp` : visualisation et validation en 2D ;
- `test3D.cpp` : simulation 3D et collision de deux galaxies.